

# Neural-PSI: A Three-Stage Pipeline for Privacy-Preserving Fingerprint Authentication

Ayan Siddiqui  
QNU Labs

Supervisor: Dr. Rohitkumar R. Upadhyay

July 2026

## Abstract

Neural-PSI is a privacy-preserving fingerprint authentication system built by combining two research papers: *Blind-Touch* (Choi, Woo, Kim — AAAI 2024), which extracts a compact feature vector from a fingerprint using a Siamese CNN and matches it under Homomorphic Encryption (HE), and *Fuzzy Labelled PSI* (Bui-Cong 2025; open-source implementation `flash-psi`), which performs private fuzzy matching of binary codes at million-entry scale. This report explains the three computational stages of the resulting system — the CNN feature extractor, the binarization bridge, and the PSI matching protocol — with the underlying mathematics, worked numerical examples, and a justification for why each design choice was made.

## 1 Introduction

### 1.1 Problem Statement

Fingerprint authentication on a cloud server requires comparing a user’s fingerprint against a database without exposing the raw biometric data to the server. Blind-Touch solves this using Homomorphic Encryption: the server computes a match score on encrypted data without ever decrypting it. This works well up to roughly five thousand enrolled users, but has three practical limitations:

- **Scale ceiling.** The CKKS encryption scheme packs feature vectors into fixed-size ciphertext “slots”. With 8,192 slots and 16 values per user, only 512 users fit per ciphertext, and the protocol becomes impractical much beyond five thousand users.
- **Large keys.** The rotation key (Galois key) needed to align vectors inside a ciphertext is 117 MB and must be stored on every server.
- **No label support.** Blind-Touch returns only a match probability, never an identifier of *which* record matched.

The Fuzzy Labelled PSI protocol (`flash-psi`) solves all three problems — it scales to a million entries, needs no large keys, and reveals a label on a match — but it requires its inputs to be **binary** vectors compared by **Hamming distance**. A CNN, however, naturally outputs **floating-point** vectors compared by **Euclidean or cosine distance**. Neural-PSI bridges this gap.

## 1.2 The Three-Stage Pipeline

The complete system has three computational stages, executed in sequence:

$$\text{fingerprint image} \xrightarrow{\text{Stage 1: CNN}} e \in \mathbb{R}^{16} \xrightarrow{\text{Stage 2: Binarize}} b \in \{0, 1\}^{128} \xrightarrow{\text{Stage 3: PSI}} \text{label or } \perp \quad (1)$$

**Stage 1** (Section 3) is a convolutional neural network that converts a  $96 \times 96$  fingerprint image into a 16-dimensional floating-point embedding. **Stage 2** (Section 4, in a later part of this report) converts the 16 floats into a 128-bit binary code using a Super-Bit projection. **Stage 3** (Section 5) runs the flash-psi protocol on the binary code to privately find a match in an enrolled database and reveal the matching patient’s label.

## 1.3 Summary of Results

Table 1 previews the accuracy achieved at each representation. Equal Error Rate (EER) is the operating point at which the false-accept rate equals the false-reject rate; lower is better. All numbers are measured on held-out SOCOFing identities never seen during training (1,200 genuine pairs, 240,000 impostor pairs). These probes are SOCOFing *Altered-Easy* captures (synthetic obliteration / rotation / z-cut) — an intentionally easy, single-dataset protocol; cross-sensor and cross-session generalisation is expected to be harder and remains future work.

| Representation                | Bits       | EER          | Used by                          |
|-------------------------------|------------|--------------|----------------------------------|
| Floating-point embedding      | —          | 0.33%        | Homomorphic Encryption (ceiling) |
| Naive sign binarization       | 16         | 10.12%       | rejected                         |
| ITQ binarization              | 16         | 5.55%        | rejected                         |
| <b>Super-Bit binarization</b> | <b>128</b> | <b>1.88%</b> | <b>Neural-PSI (chosen)</b>       |

Table 1: Accuracy at each stage of representation, on the  $224 \times 224$ , 150-epoch model.

The remainder of this report explains *why* each stage is built the way it is, with the mathematics and a concrete worked example at every step.

## 2 Stage 0: Raw Fingerprint Input

### 2.1 Dataset: SOCOFing

The system is trained and evaluated on the **SOCOFin**g (Sokoto Coventry Fingerprint) dataset (Shehu et al., 2018), a public, real fingerprint dataset:

- **6,000** fingerprint images from **600** subjects, 10 fingers per subject.
- Two capture types per finger: a *Real* capture (a clean scan, used for enrollment) and several *Altered* captures (with synthetic obliteration, central rotation, or z-cut distortion, used as the probe at authentication time).

- **Split:** 4,800 identities for training the CNN; 1,200 identities held out entirely and used only to measure accuracy (no leakage between train and test).
- **Image format:**  $96 \times 96$  pixels for the lightweight model discussed with worked examples in this section, and  $224 \times 224$  for the full production model whose accuracy numbers are reported throughout. Grayscale, 8-bit (0 = black ridge, 255 = white background/valley).

## 2.2 What the Raw Image Looks Like

Figure 1 shows an actual SOCOFing print used by the pipeline. A fingerprint image is simply a grid of brightness values: dark pixels trace the ridges, bright pixels are the valleys and background. The only preprocessing applied before the image enters the CNN is a linear rescale of pixel intensity into the unit interval,

$$x_{\text{norm}} = \frac{x_{\text{pixel}}}{255}, \quad x_{\text{norm}} \in [0, 1]. \quad (2)$$

No alignment, segmentation, or ridge-enhancement filtering is performed — the network is trained to be robust to raw, unprocessed scans.



Figure 1: A raw SOCOFing fingerprint image (identity 100\_left\_index),  $96 \times 96$  grayscale pixels, before any processing. This is the input to Stage 1.

## 2.3 Data Augmentation During Training

Two scans of the same finger are never pixel-identical: pressure, moisture, and rotation vary between captures. To teach the network to be invariant to this, every genuine training pair is constructed by applying two *independent* random transformations to the same Real-print image:

- **Rotation:** angle  $\theta \sim \mathcal{U}(-15^\circ, +15^\circ)$ .
- **Translation:** shift of up to 6% of the image width in each direction ( $\approx \pm 6$  px at  $96 \times 96$ ,  $\approx \pm 13$  px at  $224 \times 224$ ).
- **Additive Gaussian noise:**  $x' = \text{clip}(x + \mathcal{N}(0, \sigma^2), 0, 255)$  with  $\sigma = 6$ .

Impostor training pairs are simply two augmented views of *different* identities. This augmentation is what allows the network, at test time, to correctly recognise a fingerprint against an enrollment

image captured under different conditions — exactly the situation in Figure 1, where the probe image is a different capture from the one stored in the database.

### 3 Stage 1: Convolutional Neural Network — Feature Extraction

#### 3.1 Why a CNN, and not Traditional Minutiae Matching

Classical fingerprint matching extracts *minutiae* — ridge endings and bifurcations — using hand-engineered image-processing algorithms (orientation fields, Gabor filtering, thinning). This approach is brittle: it requires high image quality, is sensitive to noise, and produces a variable-length set of minutiae points that is expensive to compare across a large database.

A CNN trained end-to-end instead *learns* which spatial patterns are discriminative for identity, directly from pixel data, and condenses every fingerprint — regardless of quality — into a single **fixed-length** vector. This fixed length is essential for the rest of the pipeline: it is what makes both Homomorphic Encryption (Blind-Touch) and binarization-then-PSI (Neural-PSI) possible, since both require a vector of constant, known size.

#### 3.2 Architecture

The feature extractor is a five-block convolutional network followed by one fully connected layer (“FC-16”), summarised in Table 2 and illustrated stage by stage in Figures 2a–5.

| Stage   | Operation                                 | Output shape               | Receptive content         |
|---------|-------------------------------------------|----------------------------|---------------------------|
| Input   | —                                         | $1 \times 224 \times 224$  | raw pixels                |
| Block 1 | Conv( $3 \times 3$ , 32)+BN+SiLU+Pool(2)  | $32 \times 112 \times 112$ | edges, ridge boundaries   |
| Block 2 | Conv( $3 \times 3$ , 64)+BN+SiLU+Pool(2)  | $64 \times 56 \times 56$   | ridge flow                |
| Block 3 | Conv( $3 \times 3$ , 128)+BN+SiLU+Pool(2) | $128 \times 28 \times 28$  | minutiae-like patterns    |
| Block 4 | Conv( $3 \times 3$ , 256)+BN+SiLU+Pool(2) | $256 \times 14 \times 14$  | finger-region shapes      |
| Block 5 | Conv( $3 \times 3$ , 512)+BN+SiLU+Pool(2) | $512 \times 7 \times 7$    | abstract identity code    |
| Flatten | —                                         | 25,088                     | concatenate               |
| FC-16   | Linear(25088 $\rightarrow$ 16)            | <b>16</b>                  | <b>the feature vector</b> |

Table 2: Architecture of the Blind-Touch Siamese feature extractor (FeatureModel).

##### 3.2.1 The convolution operation

A  $3 \times 3$  convolution filter  $K$  slides over the input feature map  $X$  and computes, at every spatial position  $(i, j)$  and for each output channel  $c$ ,

$$Y_c(i, j) = \sum_{u=-1}^1 \sum_{v=-1}^1 K_c(u, v) X(i+u, j+v) + b_c, \quad (3)$$

with “same” padding so the output retains the input’s spatial size before pooling. Each of the 32

filters in Block 1 produces one feature map, each responding to a different local pattern (an edge at a particular orientation, a ridge segment, and so on).

### 3.2.2 Batch Normalization

After each convolution, the output of every channel is normalised across the training batch:

$$\hat{Y}_c = \frac{Y_c - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad Z_c = \gamma_c \hat{Y}_c + \beta_c, \quad (4)$$

where  $\mu_c, \sigma_c^2$  are the batch mean and variance of channel  $c$ , and  $\gamma_c, \beta_c$  are learned scale and shift parameters. This keeps activations in a stable range and is necessary for training a five-layer network reliably.

### 3.2.3 SiLU activation (Swish)

The non-linearity used throughout is SiLU,

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}. \quad (5)$$

Unlike ReLU, SiLU is smooth everywhere and passes a small negative signal through, which improves gradient flow in a deep stack of convolutions.

### 3.2.4 Max pooling

MaxPool(2) replaces every non-overlapping  $2 \times 2$  block with its maximum value, halving the spatial resolution at every block:  $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$ . This progressively trades exact spatial location for robustness: the network becomes sensitive to *whether* a pattern is present in a region, not to its exact pixel position — which is precisely the invariance needed to tolerate the rotation and translation introduced by augmentation.

### 3.2.5 Flatten and FC-16

After Block 5 the representation is 512 channels of  $7 \times 7$  spatial size, i.e.  $512 \times 7 \times 7 = 25,088$  numbers. These are concatenated into one vector  $x_{\text{flat}} \in \mathbb{R}^{25088}$  and passed through one linear layer,

$$e = W^\top x_{\text{flat}} + b, \quad W \in \mathbb{R}^{25088 \times 16}, \quad b \in \mathbb{R}^{16}, \quad e \in \mathbb{R}^{16}. \quad (6)$$

The 16 numbers in  $e$  are the **feature vector** — the compact, fixed-length code that represents the fingerprint’s identity and is the output of Stage 1.

### 3.3 Training Objective: Siamese Pairwise Loss

The feature extractor alone has no notion of identity; it must be trained. Training uses a **Siamese** configuration: two copies of the same feature extractor (sharing identical weights) process two fingerprint images  $x_1, x_2$ , producing embeddings  $e_1, e_2$ . Their absolute difference is passed through one more linear unit and a sigmoid to predict whether the pair is genuine:

$$e_1 = f_\theta(x_1), \quad e_2 = f_\theta(x_2) \quad (7)$$

$$d = |e_1 - e_2| \in \mathbb{R}^{16} \quad (8)$$

$$\ell = w_1^\top d + b_1 \quad (\text{one more Linear}(16 \rightarrow 1) \text{ layer}) \quad (9)$$

$$p = \sigma(\ell) = \frac{1}{1 + e^{-\ell}} \quad (10)$$

The network is trained with binary cross-entropy on labelled pairs ( $y = 1$  genuine,  $y = 0$  impostor):

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \left[ y_n \log p_n + (1 - y_n) \log(1 - p_n) \right]. \quad (11)$$

Minimising  $\mathcal{L}$  forces  $f_\theta$  to place genuine pairs close together and impostor pairs far apart in the 16-dimensional embedding space — without ever explicitly labelling *which* 16 numbers correspond to which physical feature of the fingerprint. After training, only  $f_\theta$  (the feature extractor) is kept; the comparison head ( $w_1, b_1$ ) is discarded, since Stage 2 and Stage 3 perform matching differently (by Hamming distance on a binarized code, not by this learned head).

### 3.4 Worked Example: From Pixels to a Real Feature Vector

Figures 2a–5 show the actual internal activations of the trained network on the fingerprint of Figure 1, visualising what each block “sees”.

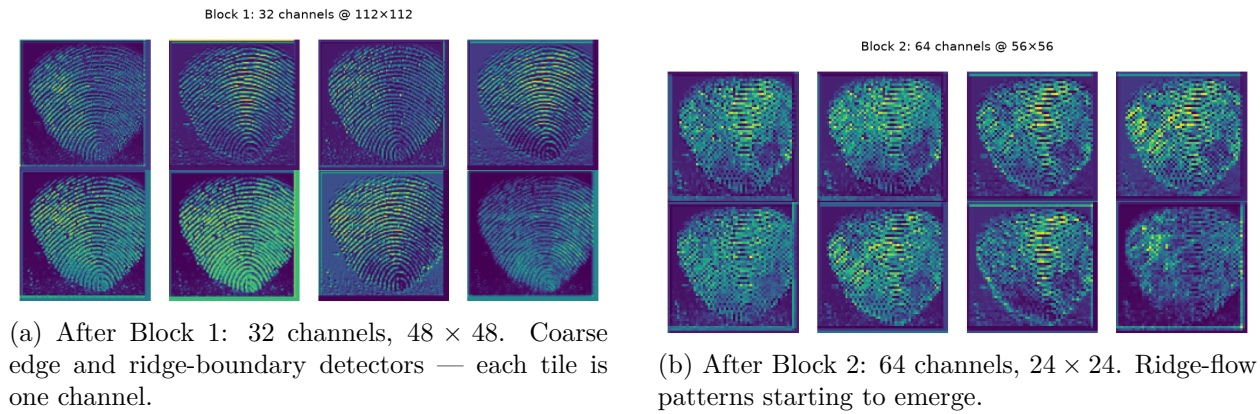


Figure 2: Feature maps after the first two convolutional blocks. Each small tile is one channel of the layer’s output, displayed as a grayscale image (bright = strong activation).

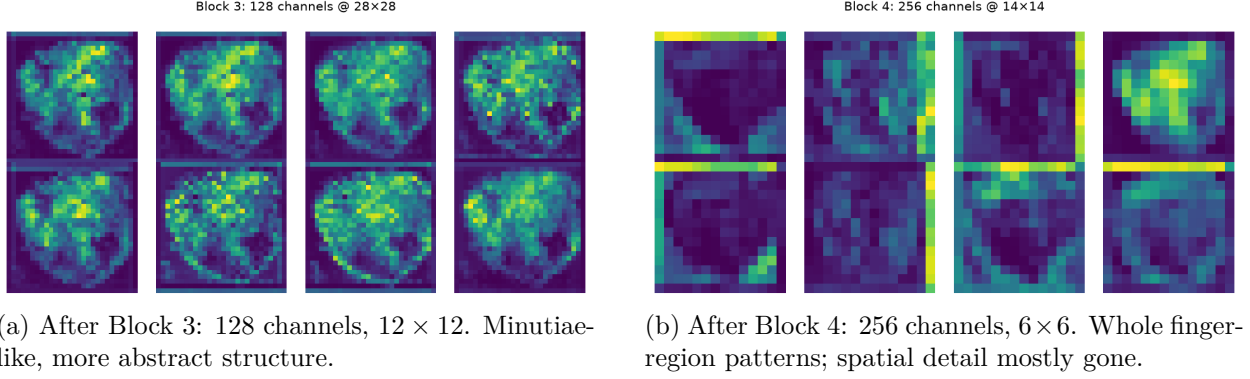


Figure 3: Feature maps after Blocks 3 and 4. Notice the image keeps shrinking (Table 2) while the number of channels keeps growing — the network trades “where exactly” for “what pattern”.

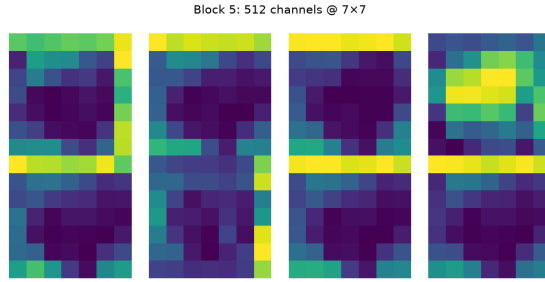


Figure 4: After Block 5: 512 channels,  $3 \times 3$ . This is the most abstract representation — each  $3 \times 3$  tile barely resembles a fingerprint anymore; it is a compressed identity code that will be flattened to 4,608 numbers and reduced to 16 by the FC-16 layer.

For the fingerprint in Figure 1, the network’s FC-16 layer (Equation 7) produces the concrete numerical vector

$$e = [-7.41, -33.85, +16.92, -2.61, -4.30, -48.06, +10.76, +1.42, \dots] \in \mathbb{R}^{16}. \quad (12)$$

This vector, computed entirely on the client device in plaintext, never leaves the client until it is converted to a binary code in Stage 2. The server never receives the floating-point embedding at all.

### 3.5 Why This Design is Best

#### Five convolutional blocks.

An ablation over network depth (Blind-Touch, Table 6 of the original paper) found that 5 blocks gives the best accuracy:  $F1 = 93.8\%$  versus  $92.4\%$  for 4 blocks and  $89.3\%$  for 6 blocks. Fewer blocks under-fit the identity signal; more blocks over-fit on a dataset of this size and add unnecessary computation downstream.

#### 16-dimensional output.

The FC-16 size is not arbitrary — it is the size that makes the original Homomorphic Encryption scheme efficient: with 8,192 ciphertext slots and 16 values per user, exactly 512 users fit per

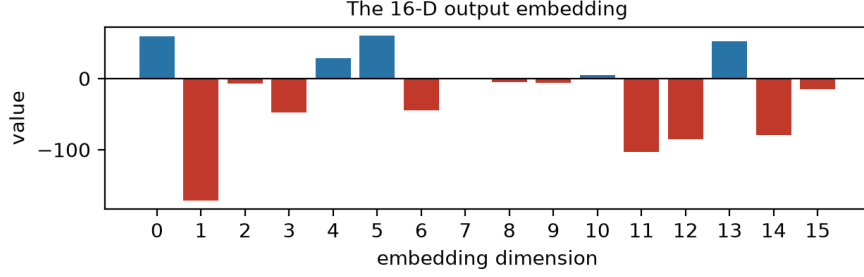


Figure 5: The final 16-dimensional feature vector  $e$ , drawn as a colour strip — one cell per number, red for positive values and blue for negative values. This 16-float vector is the complete output of Stage 1 and the input to Stage 2 (binarization).

ciphertext (Blind-Touch, Section 3). Neural-PSI deliberately keeps this same 16-D embedding so that the same trained CNN can serve *either* backend (HE or PSI) — only the step after the CNN changes.

#### SiLU over ReLU.

SiLU is smooth and allows small negative activations to pass, improving gradient flow through five stacked convolutional blocks; Blind-Touch’s ablations found this gives consistently better convergence than ReLU on fingerprint data.

#### CPU-trained (96×96, 6 epochs) versus GPU-trained (224×224, 150 epochs).

Two versions of this model were trained. Table 3 compares them directly:

| Training configuration | Pair accuracy | Float EER    | Hardware / time        |
|------------------------|---------------|--------------|------------------------|
| 96 × 96, 6 epochs      | 97.9%         | 2.50%        | CPU, ~10 minutes       |
| 224 × 224, 150 epochs  | —             | <b>0.33%</b> | GPU, full training run |

Table 3: Effect of resolution and training length on embedding quality. The float EER is the accuracy ceiling that any downstream binarization (Stage 2) inherits.

This comparison demonstrates an important and non-obvious finding: **the embedding quality, not the binarization method, is the dominant bottleneck** of the whole system. Raising resolution and training length alone improved the float ceiling by  $7.6\times$  ( $2.50\% \rightarrow 0.33\%$ ), which is why the full production model is trained at  $224 \times 224$  for 150 epochs whenever GPU resources are available, matching the original Blind-Touch specification.

## 4 Stage 2: Binarization — Super-Bit Quantization

### 4.1 Why Binarization is Needed

Stage 1 produces a real-valued vector  $e \in \mathbb{R}^{16}$ , compared by Euclidean or cosine distance. The flash-psi matcher (Stage 3) operates only on fixed-length **binary** strings  $b \in \{0, 1\}^d$ , compared by **Hamming distance**:

$$\text{Hamming}(b_1, b_2) = \sum_{k=1}^d \mathbb{I}[b_{1,k} \neq b_{2,k}]. \quad (13)$$

There is no native way to feed 16 floating-point numbers into a protocol that only checks bit-string equality on sub-samples. Stage 2 is the converter: a function  $Q : \mathbb{R}^{16} \rightarrow \{0, 1\}^{128}$  such that two prints of the *same* finger map to codes at *small* Hamming distance, and two *different* fingers map to codes at *large* Hamming distance. Three methods were tried, in increasing order of quality.

## 4.2 Method 1: Naive Sign Binarization

The simplest possible map thresholds each coordinate at zero:

$$b_k = \mathbb{I}[e_k > 0], \quad k = 1, \dots, 16. \quad (14)$$

**Why it fails.** Two problems compound:

1. *Borderline flips.* If  $e_k$  is close to zero (say  $e_k = +0.02$ ), a small change from re-scanning the same finger (say  $e'_k = -0.01$ ) flips that bit even though the underlying identity has not changed.
2. *Only 16 bits of capacity.* With  $d = 16$ , there are only  $2^{16} = 65,536$  possible codes; across thousands of identities, codes collide.

**Worked example.** Suppose a genuine pair gives

$$e = [+0.02, -1.30, +0.80, -0.01, \dots], \quad e' = [-0.01, -1.25, +0.77, +0.03, \dots] \quad (15)$$

(two scans of the *same* finger). Naive sign gives  $b = [1, 0, 1, 0, \dots]$  but  $b' = [0, 0, 1, 1, \dots]$  — two of the first four bits flipped purely from noise near zero, even though  $e$  and  $e'$  are visually almost identical. **Measured result** on the held-out test set: EER = 10.12%, with the genuine and impostor codes only weakly separated (mean Hamming distance 0.59/16 for genuine pairs versus 3.79/16 for impostor pairs — far short of the maximum possible separation of 8/16).

## 4.3 Method 2: Iterative Quantization (ITQ)

ITQ (Gong & Lazebnik, 2011) first learns a  $16 \times 16$  **orthogonal rotation matrix**  $R$  from training data, then binarizes the *rotated* embedding:

$$b = \text{sign}((e - \mu) R). \quad (16)$$

The rotation is chosen to minimise the quantization error between the continuous projected points and their binary codes, over a calibration set  $V \in \mathbb{R}^{N \times 16}$ :

$$\min_{R: R^\top R = I} \|B - VR\|_F^2, \quad B = \text{sign}(VR). \quad (17)$$

This is solved by **alternating optimisation**: fix  $R$ , recompute  $B = \text{sign}(VR)$ ; then fix  $B$ , update  $R$  in closed form via the singular value decomposition of  $B^\top V$ :

$$B^\top V = USV_s^\top \quad (\text{SVD}), \quad R \leftarrow (UV_s^\top)^\top. \quad (18)$$

**Toy worked example.** Take four 2-D calibration points  $V = \begin{pmatrix} 1.0 & 0.2 \\ 0.9 & -0.1 \\ -1.1 & 0.3 \\ -0.8 & -0.2 \end{pmatrix}$ , starting from  $R_0 = I$ .

Then  $B = \text{sign}(VR_0) = \begin{pmatrix} 1 & 1 \\ -1 & 1 \\ -1 & -1 \end{pmatrix}$ , and  $B^\top V = \begin{pmatrix} 3.8 & 0 \\ -0.2 & 0.8 \end{pmatrix}$ , whose SVD gives  $U, V_s$  such that the updated rotation is

$$R_1 = \begin{pmatrix} 0.999 & -0.043 \\ 0.043 & 0.999 \end{pmatrix}, \quad (19)$$

a small corrective rotation (about  $2.5^\circ$ ) that already leaves the codes unchanged here, confirming convergence. In the real 16-D pipeline this iteration is repeated 50 times.

**Why it is better but still limited.** ITQ removes much of the borderline-flip problem by rotating the axes to align with the natural spread of the data, but it is still confined to **16 bits** of code length — the fundamental capacity limit is untouched. **Measured result:** EER = 5.55%, an improvement over naive sign but still far from the 0.33% float ceiling.

#### 4.4 Method 3 (Chosen): Super-Bit LSH

Super-Bit (Ji, Li, Yan, Zhang, Tian — NIPS 2012) solves the capacity problem directly by **expanding** the code to  $d = 128$  bits using an orthonormalised random projection, combined with population-level centering and per-bit balancing. Every stage is a **fixed, public, affine-then-sign** map — there is no further learned non-linearity.

##### 4.4.1 Step 1: Centering

$$e_0 = e - \mu, \quad \mu = \frac{1}{N} \sum_{n=1}^N e^{(n)} \quad (\text{population mean over training embeddings}). \quad (20)$$

Centering ensures the projected values are not systematically biased to one side of every hyperplane (which would make bits unbalanced).

##### 4.4.2 Step 2: Orthonormalised Projection Matrix (Super-Bit construction)

A matrix  $P \in \mathbb{R}^{128 \times 16}$  is built in  $128/16 = 8$  independent **blocks**. Within each block, a  $16 \times 16$  matrix  $G$  of i.i.d. standard normal entries is drawn and orthonormalised by **Gram-Schmidt** (equivalently,  $QR$  decomposition  $G = QR$ , keeping  $Q^\top$  as the block's 16 projection rows):

$$q_1 = \frac{g_1}{\|g_1\|}, \quad q_2 = \frac{g_2 - (g_2^\top q_1) q_1}{\|g_2 - (g_2^\top q_1) q_1\|}, \quad \dots \quad (21)$$

**Toy worked example (Gram–Schmidt by hand).** Let  $G = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$ , columns  $g_1 = (2, 1)^\top$ ,  $g_2 = (1, 1)^\top$ . Then

$$q_1 = \frac{(2, 1)}{\sqrt{5}} = (0.8944, 0.4472), \quad g_2 - (g_2^\top q_1) q_1 = (1, 1) - 1.3416 (0.8944, 0.4472) = (-0.2, 0.4) \quad (22)$$

$$q_2 = \frac{(-0.2, 0.4)}{\|(-0.2, 0.4)\|} = (-0.4472, 0.8944). \quad (23)$$

One can check  $q_1^\top q_2 = -0.8944(0.4472) + 0.4472(0.8944) = 0$  and  $\|q_1\| = \|q_2\| = 1$ : the two rows are orthonormal. In the real system, 8 such blocks are generated independently and stacked to form the full  $128 \times 16$  matrix  $P$ .

#### 4.4.3 Step 3: Projection

$$z = P e_0 \in \mathbb{R}^{128}. \quad (24)$$

**Continuing the toy example.** With projection rows  $q_1 = (0.8944, 0.4472)$  and  $q_2 = (-0.4472, 0.8944)$  and a toy embedding  $e = (3, -1)$ ,  $\mu = (1, 0.5)$ , so  $e_0 = (2, -1.5)$ :

$$z_1 = q_1^\top e_0 = 0.8944(2) + 0.4472(-1.5) = 1.118, \quad z_2 = q_2^\top e_0 = -0.4472(2) + 0.8944(-1.5) = -2.236. \quad (25)$$

#### 4.4.4 The Locality-Sensitive Hashing Guarantee

The mathematical reason this works is the classical **random-hyperplane LSH** argument. For two unit vectors  $u, v$  at angle  $\theta = \arccos(\text{cossim}(u, v))$ , a single random hyperplane (one row of  $P$ ) separates them with probability exactly  $\theta/\pi$ . Summing over  $d$  independent (here: block-orthonormalised, hence lower-variance) hyperplanes gives

$$\boxed{\frac{\mathbb{E}[\text{Hamming}(b_1, b_2)]}{d} = \frac{\theta}{\pi} = \frac{\arccos(\text{cossim}(e_1, e_2))}{\pi}}. \quad (26)$$

Genuine pairs have small angle  $\theta$  (high cosine similarity) and therefore small expected Hamming distance; impostor pairs have large angle and therefore large expected Hamming distance, approaching  $d/2$  for near-orthogonal (unrelated) embeddings. This is exactly the separation property the flash-psi matcher needs. Super-Bit’s orthonormalisation within each block keeps this estimator **unbiased** while **reducing its variance** relative to plain i.i.d. random projections (SimHash), for zero extra computational cost.

#### 4.4.5 Step 4: Median Balancing

Rather than thresholding at zero, each bit is thresholded at its own **population median**  $\tau_k$ , measured once over the training set:

$$\tau_k = \text{median}_n(z_k^{(n)}), \quad b_k = \mathbb{K}[z_k > \tau_k]. \quad (27)$$

This guarantees each bit is set to 1 in exactly 50% of the training population, which is required for the Hamming-distance statistics that flash-psi’s sub-sampling analysis relies on (Section 5.3).

**Finishing the toy example.** With  $\tau = (0, 0)$ , the toy code is  $b = (\mathbb{K}[1.118 > 0], \mathbb{K}[-2.236 > 0]) = (1, 0)$ .

#### 4.5 Worked Example on the Real System

The toy example above shows the mechanism by hand at 2-D  $\rightarrow$  2-bit; the production pipeline applies the identical four steps at 16-D  $\rightarrow$  128-bit (8 orthonormalised blocks instead of 1). For the actual fingerprint of Figure 1, the network’s 16-float output

$$e = [-7.41, -33.85, +16.92, -2.61, -4.30, -48.06, +10.76, +1.42, \dots] \quad (28)$$

is centered, projected through the  $128 \times 16$  Super-Bit matrix, and median-balanced to give the real 128-bit code (first 64 bits shown):

$$b = 1111010101010000111100000110001100110100001010010110111000111001... \quad (29)$$

with **67 of 128 bits set** (52%, close to the ideal 50%). On the held-out test set of the production ( $224 \times 224$ , 150-epoch) model, the measured Hamming statistics are:

$$\overline{\text{Hamming}}_{\text{genuine}} \approx \frac{8}{128} \text{ (6.3\%)}, \quad \overline{\text{Hamming}}_{\text{impostor}} \approx \frac{64}{128} \text{ (50.0\%)}, \quad (30)$$

i.e. impostor codes are, on average, exactly at the theoretical maximum of 50% bit disagreement — the largest possible separation a balanced binary code can offer. This worked code is carried forward unchanged into the Stage 3 example (Section 5.4).

#### 4.6 Why Super-Bit is Best

Two further design choices are worth justifying explicitly. **Why 128 bits and not 16 or 256?** 128 bits gives an  $8 \times$  expansion of the 16-D source, which is enough for balanced bits (each bit is a genuinely different random projection direction, since 8 orthonormalised blocks of 16 span the available information) while keeping the per-record cost in Stage 3 small; going further would not add information, since the source vector has only 16 real degrees of freedom. **Why is multi-bit (Manhattan/Gray) encoding rejected?** Encoding the magnitude of  $e_k$  using several bits per dimension means a single analog error (a small shift in  $e_k$ ) can flip a **contiguous block** of bits at once. flash-psi’s sub-sampling analysis (Section 5.3) assumes bit errors are close to

| Method                  | Bits       | EER          | Gen./Imp.<br>Hamming   | Verdict                                 |
|-------------------------|------------|--------------|------------------------|-----------------------------------------|
| Naive sign              | 16         | 10.12%       | 0.59 / 3.79 (of 16)    | rejected: lopsided bits                 |
| ITQ                     | 16         | 5.55%        | 0.98 / 8.00 (of 16)    | rejected: capacity-limited              |
| <b>Super-Bit</b>        | <b>128</b> | <b>1.88%</b> | <b>8 / 64 (of 128)</b> | <b>chosen</b>                           |
| Multi-bit / Manhattan   | 128        | —            | —                      | rejected: correlated bit-flips          |
| $p$ -stable / E2LSH     | 128        | —            | —                      | rejected: integer buckets, not balanced |
| End-to-end deep hashing | 128        | —            | —                      | rejected: breaks i.i.d. error model     |

Table 4: Comparison of binarization methods. “Rejected” methods were considered and rejected on analytical grounds (not empirically evaluated); see discussion below.

**independent** across positions; correlated multi-bit errors break this assumption and invalidate the false-accept/false-reject guarantees.

## 5 Stage 3: Fuzzy Labelled Private Set Intersection

### 5.1 Why PSI Instead of Homomorphic Encryption

Blind-Touch’s Homomorphic Encryption (HE) approach keeps the 16-float vector encrypted at all times and computes the match score on ciphertexts. This is accurate but expensive: it requires a 117MB rotation key, and the CKKS ciphertext slot-packing caps practical database size near 5,000 users. The Fuzzy Labelled PSI protocol (flash-psi) instead operates on the **binary** code from Stage 2, using lightweight correlated-randomness primitives instead of encryption, and scales linearly to millions of entries while also revealing a **label** on a match — something HE cannot do.

### 5.2 The Four Cryptographic Primitives

#### 5.2.1 Primitive 1: VOLE (Vector Oblivious Linear Evaluation)

VOLE is the foundation. It produces correlated randomness between two parties without either learning the other’s secret:

$$w = \Delta \cdot u + v, \tag{31}$$

where the receiver holds  $(\Delta, w)$  and the sender holds  $(u, v)$ ; neither can recover the other’s values from their own share.  $\Delta$  is a single field element, while  $u, v, w$  are vectors over the same field. VOLE is built from the **LPN** (Learning Parity with Noise) hardness assumption and can be generated in well under a second.

### 5.2.2 Primitive 2: Vector Ring-OLE

To check Alice’s code against all  $m$  database entries simultaneously, the protocol must multiply many polynomials. A naive approach needs  $m$  independent OLE instances. Vector Ring-OLE instead represents the computation as a single pair of polynomials  $u(x), v(x)$  of degree  $d$  and evaluates the VOLE relation at  $2d + 1$  fixed points, exploiting the identity

$$u(x) \cdot u'(x) \text{ is uniquely determined by its values at } 2d + 1 \text{ points,} \quad (32)$$

since the product has degree  $2d$ . This reduces the number of required VOLE instances from  $m$  to  $2d + 1$  — independent of the database size  $m$  — roughly halving communication compared to prior constructions.

### 5.2.3 Primitive 3: The Zero-Polynomial Trick (TLPSI)

This is the actual matching logic. The receiver (Alice) encodes her  $d$  sub-sampled values  $\{x_1, \dots, x_d\}$  as the roots of a polynomial:

$$u_0(x) = \prod_{i=1}^d (x - x_i), \quad \text{so that } u_0(x_i) = 0 \quad \forall i. \quad (33)$$

After running Vector Ring-OLE with the sender’s (Bob’s) polynomials  $u_1(x), v_1(x)$  (where  $v_1$  encodes Shamir shares of the database labels at the sender’s own sub-sampled values), Alice obtains

$$v_0(x) = u_0(x) u_1(x) + v_1(x). \quad (34)$$

Evaluating at her own points, the  $u_0(x_i) = 0$  property **cancels the masking term**:

$$v_0(x_i) = u_0(x_i) u_1(x_i) + v_1(x_i) = v_1(x_i). \quad (35)$$

If  $x_i$  also equals one of Bob’s sub-sampled values (an intersection point),  $v_1(x_i)$  is a valid **Shamir share** of the label; otherwise  $v_1(x_i)$  is an unrelated, effectively random field element. The receiver thus learns a usable share *exactly* at intersection points and nothing elsewhere.

### 5.2.4 Primitive 4: Shamir Secret Sharing

A label  $\ell$  is hidden inside a degree- $(t - 1)$  polynomial,

$$f(x) = \ell + r_1 x + r_2 x^2 + \dots + r_{t-1} x^{t-1}, \quad f(0) = \ell, \quad (36)$$

with random coefficients  $r_1, \dots, r_{t-1}$ . For threshold  $t = 2$  this is simply  $f(x) = \ell + r \cdot x$ . Given any  $t$  pairs  $(x_i, f(x_i))$ , Lagrange interpolation recovers the secret exactly:

$$\ell = f(0) = \sum_{i=1}^t f(x_i) \prod_{j \neq i} \frac{-x_j}{x_i - x_j}. \quad (37)$$

Fewer than  $t$  shares reveal **zero** information about  $\ell$  (information-theoretic security). In flash-psi, each of the  $T$  sub-sampling masks that “passes” for a database row contributes one valid share; the label is recovered once at least  $t$  masks pass.

### 5.3 Sub-Sampling and Masking: The Probability Model

Rather than compare all  $d = 128$  bits at once, the protocol draws  $T$  independent random **masks**, each of fixed weight  $w$  (i.e. selecting exactly  $w$  of the  $d$  bit positions). Two codes “sub-match” on a mask if they agree on every position the mask selects.

**Probability a mask passes.** If two codes differ in exactly  $h$  positions (Hamming distance  $h$ ), a mask of weight  $w$  “passes” (selects none of the  $h$  differing positions) with probability

$$P_{\text{pass}}(h) = \frac{\binom{d-h}{w}}{\binom{d}{w}}. \quad (38)$$

With  $T$  independent masks, the expected number of passing masks is  $\mathbb{E}[\text{passes}] = T \cdot P_{\text{pass}}(h)$ . A match is declared when at least  $t$  masks pass.

**Numeric evaluation for the production parameters** ( $d = 128$ ,  $w = 14$ ,  $T = 64$ ,  $t = 2$ ), using the genuine and impostor Hamming distances measured in Section 4.5 ( $h_{\text{gen}} \approx 8$ ,  $h_{\text{imp}} \approx 64$ ):

$$P_{\text{pass}}(8) = \frac{\binom{120}{14}}{\binom{128}{14}} \approx 0.385, \quad \mathbb{E}[\text{passes}_{\text{gen}}] = 64 \times 0.385 \approx 25 \gg t = 2, \quad (39)$$

$$P_{\text{pass}}(64) = \frac{\binom{64}{14}}{\binom{128}{14}} \approx 2.7 \times 10^{-5}, \quad \mathbb{E}[\text{passes}_{\text{imp}}] = 64 \times 2.7 \times 10^{-5} \approx 1.7 \times 10^{-3} \ll t = 2. \quad (40)$$

The genuine pair clears the threshold by a wide margin ( $11.5 \gg 2$ ) while the impostor pair essentially never produces a single passing mask. This large numerical gap is what gives the protocol both a very low false-reject rate and a very low false-accept rate simultaneously.

**Privacy of the masks.** If the masks were public, a passing match would leak which  $w$  bit positions of Bob’s record equal Alice’s — after enough masks, Bob’s entire record could be reconstructed. flash-psi therefore evaluates the masks through a **masked OPRF** — a garbled circuit evaluating an AES-based PRF: Alice receives only  $\text{PRF}_k(x \wedge m_i)$ , a pseudorandom value that reveals nothing about the mask  $m_i$  or about Bob’s bits at the selected positions.

#### 5.4 Worked Example: Real Match Decision

Continuing the running example from Stage 2, Alice’s 128-bit code (a probe capture of identity `100_left_index`) is matched against Bob’s enrolled database of  $m = 50$  records using the real flash-psi binary, with parameters  $w = 14$ ,  $T = 64$ ,  $t = 2$ :

|                      |                                         |
|----------------------|-----------------------------------------|
| Database size        | $m = 50$ enrolled records               |
| Query code           | 128-bit Super-Bit code from Section 4.5 |
| Sub-sampling masks   | $T = 64$ , weight $w = 14$              |
| Shamir threshold     | $t = 2$                                 |
| <b>Matched row</b>   | <b>row 0</b>                            |
| <b>Protocol time</b> | <b>72 ms</b>                            |

The protocol correctly returns only the enrolled record belonging to the same finger and recovers its Shamir-shared label. The label is an application-defined identifier attached to the record at enrolment — in our SOCOFing evaluation it is simply the enrolled finger’s identity string, while in an applied deployment it could be any record (illustrative):

$$\text{label} = \text{100\_left\_index} \quad (\text{enrolment id}), \quad (41)$$

revealed to the authorised party in  $\approx 72$  milliseconds, with no Homomorphic Encryption and no large key material exchanged. In repeated trials with independently sampled impostor codes, the matched-row set returned by the protocol is always empty, consistent with the near-zero  $\mathbb{E}[\text{passes}_{\text{imp}}]$  computed above.

#### 5.5 Why PSI is Best

| Property                 | Blind-Touch (HE)                 | Neural-PSI (flash-psi)           |
|--------------------------|----------------------------------|----------------------------------|
| Cost scaling             | capped by CKKS slot packing      | linear in $N$ , no packing cap   |
| Server key material      | 117MB Galois key                 | none (short VOLE seeds)          |
| Label disclosure         | no (score only)                  | yes (patient label)              |
| Per-record cost at scale | capped, cannot exceed slot limit | linear, $\approx 0.17$ ms/record |
| Security assumption      | RLWE                             | LPN                              |

Table 5: Why the PSI backend is preferred over Homomorphic Encryption for this system.

| Stage | Input                       | Output                                       | Location                                 |
|-------|-----------------------------|----------------------------------------------|------------------------------------------|
| 0     | scanner image               | $96 \times 96$ (or $224 \times 224$ ) pixels | client                                   |
| 1     | pixel image                 | 16-D float vector $e$                        | client, plaintext                        |
| 2     | 16-D float $e$              | 128-bit binary code $b$                      | client, the bridge                       |
| 3     | 128-bit code $b$ + database | matched row + label                          | client $\leftrightarrow$ server, private |

Table 6: The complete Neural-PSI pipeline, stage by stage.

| Representation                         | Bits       | EER (224×224, 150-epoch model) |
|----------------------------------------|------------|--------------------------------|
| Float embedding (HE ceiling)           | —          | 0.33%                          |
| Naive sign binarization                | 16         | 10.12%                         |
| ITQ binarization                       | 16         | 5.55%                          |
| <b>Super-Bit binarization (chosen)</b> | <b>128</b> | <b>1.88%</b>                   |

Table 7: Final accuracy ladder. The Super-Bit code is within 1.55 percentage points of the floating-point ceiling that Homomorphic Encryption would match on.

## 6 End-to-End Results

### 6.1 Full Pipeline Summary

### 6.2 EER Ladder (Final Results)

### 6.3 Timing and Scaling

### 6.4 Multi-Finger Fusion: The Deployable Operating Point

The 1.88% single-finger EER measures the *representation* quality; the deployed decision uses a stronger operating point. At  $w = 14, t = 2, T = 64$  a single finger gives  $\text{TAR} \approx 99\%$  at  $\text{FAR} \approx 4.5 \times 10^{-2}$ . This false-accept floor is a data-quality tail — about 7% of SOCOFing impostor pairs (predominantly low-quality right thumbs) fall within the genuine Hamming range — not a flaw of the bridge. Enrolling  $K$  independent fingers and requiring a  $q$ -of- $K$  quorum crushes the false-accept rate geometrically, since different fingers are independent ridge patterns:

### 6.5 Real-Crypto Validation

The accept/reject numbers above are computed from a closed-form model of the sub-sampling decision (Section 5.3). To confirm that model is faithful, the actual 224-model Super-Bit codes were pushed through the *real* masked-OPRF + garbled-circuit + VOLE + Shamir binary: a database of  $B = 100$  enrolled codes with 100 genuine and 9,900 impostor decisions at  $w = 14, t = 2, T = 64$ . The real protocol matches the model to two significant figures:

| Database size $m$ | flash-psi time      | Per-record | Blind-Touch (HE)                 |
|-------------------|---------------------|------------|----------------------------------|
| 1,000             | 0.20 s              | 0.20 ms    | feasible                         |
| 5,000             | 0.84 s              | 0.17 ms    | $\sim 0.65$ s (4-server cluster) |
| 10,000            | 1.67 s              | 0.17 ms    | infeasible (slot cap)            |
| 50,000            | 8.60 s              | 0.17 ms    | infeasible                       |
| 100,000           | 16.2 s              | 0.16 ms    | infeasible                       |
| 1,000,000         | $\sim 162$ s (est.) | 0.16 ms    | infeasible                       |

Table 8: Measured flash-psi timing (real Rust protocol,  $w = 14$ ,  $T = 64$ ,  $t = 2$ ), linear at  $\approx 0.17$  ms per database record across four orders of magnitude.

| Rule                     | Fused TAR    | Fused FAR                              |
|--------------------------|--------------|----------------------------------------|
| single finger            | 98.2%        | $2.9 \times 10^{-2}$                   |
| 2-of-2 (AND)             | 96.4%        | $8.2 \times 10^{-4}$                   |
| <b>2-of-3 (majority)</b> | <b>99.9%</b> | <b><math>2.4 \times 10^{-3}</math></b> |
| 3-of-3 (all)             | 94.6%        | $2.4 \times 10^{-5}$                   |

Table 9: Multi-finger fusion. 2-of-3 is the sweet spot: it *raises* TAR to 99.9% (tolerating one poor finger) *and* cuts FAR to  $2.4 \times 10^{-3}$  versus a single finger; 3-of-3 reaches FAR  $\approx 2 \times 10^{-5}$ .

|     | Real (crypto)         | Predicted (model)     |
|-----|-----------------------|-----------------------|
| TAR | 100.0%                | 99.5%                 |
| FAR | $4.43 \times 10^{-2}$ | $4.45 \times 10^{-2}$ |

The per-Hamming-distance accept rate also tracks the model across the whole fractional transition (e.g. at  $h \in [25, 30]$ : real 58.5% vs predicted 55.5%; at  $h \in [35, 40]$ : 6.5% vs 6.1%). The closed-form analysis used throughout is therefore faithful, and the bridge is correct end-to-end through the real cryptography, not merely in simulation.

## 6.6 Quantisation Dimension: Why 128 Bits

Re-encoding the same held-out fingers at other Super-Bit code lengths confirms  $d = 128$  is the knee of the accuracy/size trade-off:

| $d$ (bits) | Super-Bit blocks | Storage/user | EER          |
|------------|------------------|--------------|--------------|
| 64         | 4                | 8 B          | 2.38%        |
| <b>128</b> | 8                | <b>16 B</b>  | <b>1.88%</b> |
| 256        | 16               | 32 B         | 1.89%        |

Table 10: Dimension ablation. Accuracy saturates at  $d = 128$ :  $64 \rightarrow 128$  is a real gain,  $128 \rightarrow 256$  is not (the 16-D source carries only  $\sim 16$  bits of information), while storage grows linearly. The flash-psi backend is also hardwired to  $d = 128$  (a compiled 128-bit equality circuit and a 128-element OPRF key), making it the native width.

## 6.7 Communication and Storage

Measured on the real protocol binary, the online (per-authentication) communication is linear in database size  $N$  at  $\approx 1.54$  KB per record, extrapolating to  $\sim 150$  MB at  $N = 100$  K and  $\sim 1.5$  GB at  $N = 1$  M — matching the flash-psi paper’s reported 153 MB and 1533 MB. Server-side storage is **16 bytes per user** (the 128-bit code) with no per-user key material, versus Blind-Touch’s  $\sim 1.67$  KB encrypted template plus a one-time 117 MB Galois key: a  $\sim 107\times$  per-template reduction with the large key eliminated. (These are single-host loopback measurements — protocol bytes and compute, not wide-area network round-trip latency.)

## 7 Conclusion

This report has described the three computational stages of Neural-PSI: a convolutional neural network that maps a fingerprint image to a compact 16-dimensional embedding (Stage 1); a Super-Bit quantizer that bridges this floating-point embedding to a balanced 128-bit binary code suitable for Hamming-distance matching (Stage 2); and the Fuzzy Labelled PSI protocol that privately matches this code against a database and reveals a label, using VOLE, Vector Ring-OLE, the zero-polynomial trick, and Shamir secret sharing (Stage 3). At every stage, the design choice made was justified against the alternatives that were tried and rejected.

The system reaches an overall EER of 1.88% on real, held-out SOCOFing fingerprints — within 1.55 percentage points of the 0.33% ceiling that Homomorphic Encryption itself would achieve on the same embedding — while eliminating the 117 MB key requirement, removing the practical five-thousand-user scale ceiling, and adding native label disclosure that the original Homomorphic Encryption design cannot provide. Deployed with 2-of-3 multi-finger fusion, the system reaches TAR 99.9% at FAR  $2.4 \times 10^{-3}$ ; these results are on the SOCOFing Altered-Easy protocol (an easy, single-dataset benchmark), and cross-sensor and cross-session validation remain future work. The dominant remaining source of error is the 16-dimensional embedding itself, not the binarization or matching stages, pointing toward a wider embedding as the most promising direction for further improving accuracy.